

2. 기본 문법

2-1 변수 알아보기

- 변수란
 - 변수(variable) : 프로그램에서 자료를 담아두는 공간
 - **재사용성: 한 번 저장한 값 재사용 가능**
(예) 학생의 점수를 변수에 저장하면, 그 점수를 계산하거나 출력할 때마다 다시 입력할 필요가 없음.
 - **가독성: 코드를 읽기 쉽다.**
(예) `let studentScore = 90;` 이라고 쓰면, 이 코드를 보는 사람은 `studentScore` 가 학생의 점수를 의미한다는 것을 쉽게 알 수 있음
- 변수 선언하기
 - `let` 뒤에 변수 이름 작성
 - `var`는 사용을 자제한다....
- 상수 선언하기
 - 상수는 변수의 일종. 한번 값을 할당하면 프로그램 안에서 값이 바뀌지 않음
 - `const PI=3.14;`
 -
- 변수 선언 규칙
 - 변수 이름
 - 영어 문자, 언더스코어(_), 숫자를 사용한다
 - 첫 글자는 영문자, _기호, \$기호를 사용한다
 - 띄어쓰기나 기호는 허용하지 않는다
예) `now`, `_now`, `now25` (사용할 수 있음) 예) `25now`, `now 25`, `*now` (사용할 수 없음)
 - 영어 대소문자를 구별하며 예약어는 변수 이름으로 사용할 수 없다
 - 여러 단어를 연결할 때는 하이픈이나 언더스코어를 사용할 수 있고 중간에 대문자를 섞어 쓸 수도 있다 예) `total-area`, `total_area`, `totalArea` 등

- 변수 이름은 의미있게 작성한다(네이밍 컨벤션)

2-2 자료형

종류	설명	예시
원시 유형	숫자형	따옴표 없이 숫자로만 표기합니다.
	BigInt	숫자형으로는 표현할 수 없는 아주 큰 정수로. 숫자 끝에 n을 붙여서 표시합니다.
	문자열	작은따옴표(' ')나 큰따옴표(" ")로 묶어서 나타냅니다. 숫자를 따옴표로 묶으면 문자로 인식합니다.
	논리형	참(true)과 거짓(false)이라는 2가지 값만 있는 유형입니다. 이때 true와 false는 소문자로만 표시합니다.
복합 유형	배열	하나의 변수에 값을 여러 개 저장합니다.
	객체	함수와 속성을 함께 포함합니다.
특수 유형	undefined	자료형이 지정되지 않았을 때의 상태입니다. 예를 들어 변수 선언만 하고 값을 할당하지 않은 변수는 undefined 상태입니다.
	null	값이 유효하지 않을 때의 상태입니다.

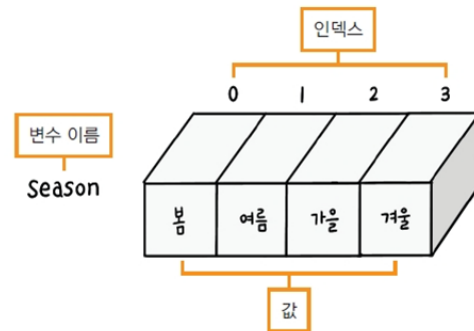
• 템플릿 리터럴(template literal)

- 문자열과 변수, 식을 섞어서 하나의 문자열을 만드는 방식
- 백틱(` `) 기호를 사용해 문자열을 만든다.
- 템플릿 리터럴 안의 변수나 식은 \${ } 로 묶어서 표현
- 태그나 띄어쓰기, 특수 기호(이스케이프 문자)는 그대로 사용 가능

```
let name = 'Kim'
let classRoom = 204
console.log(`${name}님, ${classRoom}호 강의실로 입장하세요.
`)
```

• 배열(array)

```
let season = ["봄", "여름", "가을", "겨울"];
```



• 느슨한 자료형 체크

- 자바스크립트는 미리 변수의 자료형을 지정하지 않는다.
- 변수에 값을 할당하면 그 시점에 자료형이 결정된다.
- 여러 사람이 프로젝트를 진행할 경우 변수를 일관성 있게 유지하기 힘들다.

• 자동 형 변환

- 연산하는 동안 자동으로 자료형이 바뀔 수 있다.
- 문자열 값을 사칙 연산에서 사용하면 자동으로 숫자형으로 변환됨.
- 숫자를 문자열에 연결하면 자동으로 문자열로 변환됨

```
input = prompt("숫자를 입력하세요.") // input은 문자열
input * 10 // 50. input이 숫자형으로 바뀜
```

```
let one = '20' // 문자열
let two = 10 // 숫자형
one + two // '2010'. 숫자였던 two가 문자열로 바뀜
```

• 자료형 변환 함수

함수	설명
Number()	문자열이나 논리형 값을 숫자로 변환합니다.
parseInt()	문자열을 정수 숫자로 변환합니다.
parseFloat()	문자열을 실수 숫자로 변환합니다.
String()	숫자나 논리형 값을 문자열로 변환합니다.
Boolean()	괄호 안의 값을 논리형으로 변환합니다.

2-3 연산자

- 산술 연산자

종류	설명	예시
+	두 피연산자의 값을 더합니다.	$c = a + b$
-	첫 번째 피연산자 값에서 두 번째 피연산자 값을 뺍니다.	$c = a - b$
*	두 피연산자의 값을 곱합니다.	$c = a * b$
/	첫 번째 피연산자 값을 두 번째 피연산자 값으로 나눕니다.	$c = a / b$
%	첫 번째 피연산자 값을 두 번째 피연산자 값으로 나눈 나머지를 구합니다.	$c = a \% b$
++	피연산자를 1 증가시킵니다.	$a++$
--	피연산자를 1 감소시킵니다.	$b--$

- 할당 연산자(대입)

종류	설명	예시
=	연산자 오른쪽의 값을 왼쪽 변수에 할당합니다.	y = x + 3
+=	y = y + x를 의미합니다.	y += x
-=	y = y - x를 의미합니다.	y -= x
*=	y = y * x를 의미합니다.	y *= x
/=	y = y / x를 의미합니다.	y /= x
%=	y = y % x를 의미합니다.	y %= x

- **비교연산자**

종류	설명	예시	
		조건식	결괏값
==	피연산자가 서로 같으면 true입니다.	3 == "3"	true
===	피연산자도 같고 자료형도 같으면 true입니다.	a === "3"	false
!=	피연산자가 서로 같지 않으면 true입니다.	3 != "3"	false
!==	피연산자가 같지 않거나 자료형이 같지 않으면 true입니다.	3 !== "3"	true
<	왼쪽 피연산자가 오른쪽 피연산자보다 작으면 true입니다.	3 < 4	true
<=	왼쪽 피연산자가 오른쪽 피연산자보다 작거나 같으면 true입니다.	3 <= 4	true
>	왼쪽 피연산자가 오른쪽 피연산자보다 크면 true입니다.	3 > 4	false
>=	왼쪽 피연산자가 오른쪽 피연산자보다 크거나 같으면 true입니다.	3 >= 4	false

- **문자열의 비교**

- 문자열에 있는 문자들의 ASCII 값을 비교
- 비교할 문자가 여러 개일 경우 맨 앞의 문자부터 하나씩 비교

- **논리 연산자**

종류	기호	설명
OR 연산자		피연산자 중 하나만 true여도 true가 됩니다.
AND 연산자	&&	피연산자가 모두 true일 경우에만 true가 됩니다.
NOT 연산자	!	피연산자의 반댓값을 지정합니다.

3-4. 조건문

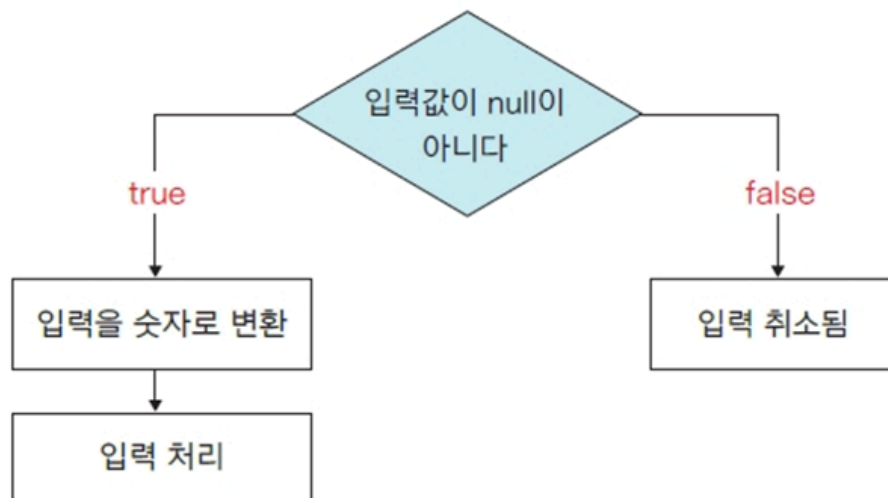
- if 문과 if ~ else 문

```
<script>
let userNumber = parseInt(prompt('숫자를 입력하세요.'));
if (userNumber % 3 === 0)
    alert('3의 배수입니다.');
```

```
else
    alert('3의 배수가 아닙니다.');
```

```
</script>
```

- 중첩된 if ~ else 문



```

<!-- 3의 배수 확인하기 -->
<script>
    let userNumber = prompt('숫자를 입력하세요. ');

    if (userNumber !== null) {
        if (parseInt(userNumber) % 3 === 0)
            console.log('3의 배수입니다. ');
        else
            console.log('3의 배수가 아닙니다. ');
    }
    else
        console.log('입력이 취소됐습니다. ');
</script>

```

- 조건 연산자로 조건 체크하기

- 삼항 연산자

기본형 (조건) ? true일 때 실행할 명령 : false일 때 실행할 명령

```

<script>
    let userNumber = prompt('숫자를 입력하세요. ');

    if (userNumber !== null) {
        (parseInt(userNumber) % 3 === 0) ? alert('3의 배수입
니다. ') : alert('3의 배수가 아닙니다. ');
    }
    else
        alert('입력이 취소됐습니다. ');
</script>

```

- switch문

기본형 switch (조건)

```
{
  case 값1: 명령1
    break
  case 값2: 명령2
    break
  .....
  default: 명령n
}
```

```
<script>
  let session = prompt("관심 세션을 선택해 주세요. 1-마케팅, 2-
개발, 3-디자인");
  switch(session) {
    case "1": document.write("<p>마케팅 세션은 <strong>201호
</strong>에서 ..... </p>")
      break;
    case "2": document.write("<p>개발 세션은 <strong>203호</s
trong>에서 ..... </p>")
      break;
    case "3": document.write("<p>디자인 세션은 <strong>205호
</strong>에서 ..... </p>")
      break;
    default: alert("잘못 입력했습니다."); // 1, 2, 3이 아닌 값
을 입력받으면 출력
  }
</script>
```

2-5 반복문

- **for 문**

기본형

```
for(초깃값; 조건; 증가식) {
  실행할 명령
}
```

1 2 3

```
//for문을 사용해 1부터 5까지 숫자 더하기
<script>
  let sum = 0;
  for(let i = 1; i < 6; i++) {
    sum += i;
  }
  document.write(`1부터 5까지 더하면 ${sum}`);
</script>
```

```
const citis = ["서울", "대구", "부산"]

for(let i = 0; i < citis.length; i++) {
  console.log(`${citis[i]}`)
}
```

- 중첩된 for 문

```
<script>
  for (let i = 1; i <= 9; i++) {
    document.write(`<h3>${i}단</h3>`);
    for (let j = 1; j <= 9; j++) {
      document.write(`${i} X ${j} = ${i * j} <br>`);
    }
  }
</script>
```

- for ~ in 문

- 객체의 열거 속성을 통해 지정된 변수를 반복

```
// for...in (객체순환)
let book = {
  title: "자바스크립트 완벽하게 이해하기",
  price: 32000,
  author: "훈이",
  publisher: "자바스크립트짱짱",
}
```

```
};

//i(변수명은 임의변경 가능) in 객체명
for (const i in book) {
  console.log(i, book[i]);
}
```

- for ...of 문

- 각각의 고유한 특성의 값을 실행할 명령과 함께 사용자 지정 반복 후크를 호출하여,
- 반복 가능한 객체(배열, Map, Set, 인수 객체 등을 포함)를 통해 반복하는 루프를 만듭니다.

```
let arr = [3, 5, 7];
arr.foo = "hello";

for (let i in arr) {
  console.log(i); // logs "0", "1", "2", "foo"
}

for (let i of arr) {
  console.log(i); // logs "3", "5", "7"
}
```

- for ...of 반복문 에서 객체사용이 불가능합니다.

- while 문

```
기본형 while (조건) {
    실행할 명령
}
```

- do ~ while 문

```
기본형 do {
    실행할 명령
} while (조건)
```

- **break 문**

- 구구단 2단~4단만 표시하기

```
<div class="container">
  <script>
    for (let i = 2; i <= 9; i++) {
      document.write(`<div class="times">`);
      document.write(`<h3>${i}단</h3>`);
      for (let j = 1; j <= 9; j++) {
        document.write(`${i} X ${j} = ${i * j} <br>`);
      }
      document.write(`</div>`);
      if (i === 4) { // 4단까지만 표시
        break;
      }
    }
  </script>
</div>
```

- **continue 문**

- 조건에 해당되는 값을 만나면 반복문의 맨 앞으로 이동
→ 결과적으로 반복 과정을 한 차례 건너 뛴

```
<script>
  let n = 10;
  let sum = 0;
  for (let i = 1; i <= n; i++) {
    if (i % 2 === 1) { // 홀수는 더하지 않고 건너뛴
      continue
    }
    sum += i;
    document.write(`${i} ----- ${sum} <br>`);
  }
</script>
```